Labo 14 — ConfigMaps et Secrets

Théorie — comment la même image sert tous les environnements dans un cluster, où mettre un mot de passe quand `-e` et `--env-file` n'existent plus, et pourquoi « base64 » n'a jamais protégé personne.

Objectifs

- Reprendre le principe du labo 08 — une image, une configuration injectée — avec les objets Kubernetes : **ConfigMap** et **Secret**.
- Injecter une configuration en **variables d'environnement** ou en **fichiers montés**, et savoir quand choisir l'un ou l'autre.
- Comprendre ce qu'un Secret protège réellement, et ce qu'il ne protège pas.
- Savoir ce qui se passe quand on modifie une ConfigMap : variables figées, fichiers mis à jour.
- Relier tout cela aux profils Spring Boot et aux gestionnaires de secrets d'entreprise.

---

1. Le problème du labo 08, à l'échelle du cluster

Au labo 08, la règle était : une image immuable, une configuration qui vient de l'extérieur, les secrets dans des fichiers plutôt que dans des variables. Les mécanismes étaient `ENV`, `-e`, `--env-file`, `--secret`. Dans le cluster, il n'y a plus de ligne de commande : tout est manifeste. Et un template de Pod (labo 12) est **le même pour tous les réplicas et pour tous les environnements** — y mettre `APP_PROFILE: prod` en dur reviendrait à reconstruire une image par environnement.

Kubernetes sépare donc la configuration du template dans deux objets : la **ConfigMap** pour ce qui n'est pas sensible (URL, profils, fichiers de configuration), le **Secret** pour ce qui donne un accès (mots de passe, jetons, certificats). Le template les **référence** par leur nom ; chaque environnement a les siens, et le Deployment ne change pas d'un namespace à l'autre.

> **→ RAPPEL DOCKER**
> ConfigMap = `--env-file` et les fichiers de configuration montés en bind mount ; Secret = `podman secret` et la convention `_FILE`. La différence : dans le cluster, ces objets sont stockés dans l'API server, versionnés avec les manifestes, et disponibles sur n'importe quel nœud où le Pod atterrit — plus de `hostPath` vers un fichier qui n'existe que sur votre machine (labo 11, question 13).

2. La ConfigMap : des clés, des valeurs, des fichiers

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
data:
  APP_MESSAGE: "bonjour depuis une ConfigMap"
  APP_PROFILE: "staging"
```

Une ConfigMap est un dictionnaire de chaînes. Une valeur peut être un mot (`staging`) ou un fichier entier (`default.conf` avec ses vingt lignes) : `kubectl create configmap web-nginx --from-`

`file=default.conf=nginx-proxy.conf` crée la clé `default.conf` avec le contenu du fichier. Elle n'est utile qu'une fois **référencée** par un Pod, de deux façons :

Injection	Syntaxe dans le template	Ce que voit le conteneur	Mise à jour de la ConfigMap
variables d'environnement	`envFrom: [configMapRef: {name: api-config}]` (toutes les clés) ou `env: [valueFrom: configMapKeyRef]` (une clé)	`APP_PROFILE=staging` dans `env`	**pas répercutée** : il faut recréer le Pod
volume	`volumes: [configMap: {name: web-nginx}]` + `volumeMounts`	un fichier par clé dans le répertoire monté	**répercutée** en une minute environ, sans redémarrage

Le choix suit le labo 08 : des variables pour ce que l'application lit au démarrage (Spring Boot, la JVM), des fichiers pour une configuration structurée ou longue (`nginx.conf`, `application.yaml`, un certificat). Dans le labo pratique, nginx est l'image officielle `nginx:alpine` **non modifiée**, configurée par deux ConfigMaps montées — l'image `web:1.0` du labo 13 n'est plus nécessaire.

PIÈGE
Une variable d'environnement est copiée dans le processus à sa création : modifier la ConfigMap ne change **rien** aux Pods en cours. `kubectl rollout restart deployment/api` les recrée avec les nouvelles valeurs. Un fichier monté, lui, est un lien symbolique vers un répertoire que le kubelet remplace lors de sa synchronisation périodique (`..data/`, visible dans `ls -l`) — sauf si le montage utilise `subPath`, qui copie le fichier une fois pour toutes. Et une clé référencée qui n'existe pas (`key: profil` au lieu de `APP_PROFILE`) bloque le Pod en `CreateContainerConfigError` : `couldn't find key profil in ConfigMap`.

→ SPRING BOOT
Les deux voies se combinent naturellement : une ConfigMap de variables (`SPRING_PROFILES_ACTIVE`, `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`) et une ConfigMap-fichier montée dans `/config/application.yaml`, que Spring Boot lit sans configuration supplémentaire. Pour les secrets, `spring.config.import=optional:configtree:/run/secrets/` transforme chaque fichier monté en propriété (`db.password` depuis `/run/secrets/db.password`). La priorité reste celle du labo 08 : variable > fichier externe > fichier dans le JAR.

3. Le Secret : la même chose, avec une étiquette « sensible »

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "S3cr3t-pg"
```

Un Secret s'écrit comme une ConfigMap (`stringData` pour du texte, `data` pour du base64) et s'injecte de la même façon : `env` avec `secretKeyRef`, ou volume `secret:` qui monte un fichier par clé. Ce qu'il ajoute : les valeurs sont **encodées en base64** dans l'API (`UzNjcjN0LXBn` pour `S3cr3t-pg`), le kubelet les monte en **tmpfs** (jamais sur le disque du nœud), les permissions RBAC

(labo 16) peuvent interdire `get secrets` à qui a le droit de lire les ConfigMaps, et les outils (`kubectl describe`, les tableaux de bord) les masquent.

> → SÉCURITÉ
> Le base64 n'est **pas** un chiffrement : `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` rend le mot de passe à quiconque peut lire le Secret. Dans etcd, les Secrets sont en clair par défaut ; les clusters sérieux activent le **chiffrement au repos** (`EncryptionConfiguration`, ou un KMS cloud) et restreignent l'accès à etcd. Un manifeste `Secret` avec sa valeur **ne va jamais dans Git** : on l'y met chiffré (SOPS, Sealed Secrets) ou on le fait générer par un contrôleur depuis un gestionnaire externe (External Secrets Operator + Vault, AWS Secrets Manager…). La règle du labo 08 — un secret visible dans un ticket est compromis — vaut pour `kubectl get -o yaml` collé dans un chat.

Les Secrets ont des types : `Opaque` (libre), `kubernetes.io/tls` (certificat + clé, consommé par l'Ingress pour le HTTPS), `kubernetes.io/dockerconfigjson` (identifiants de registry, référencé par `imagePullSecrets` pour tirer une image privée — ce que notre `k3s ctr images import` dispense de faire). Un Secret comme une ConfigMap peut être marqué `immutable: true` : plus modifiable, donc plus de risque de changement accidentel, et moins de charge pour l'API server.

4. Quatre façons de créer

Commande	Usage
`kubectl apply -f configmap.yaml`	la voie normale : un fichier versionné
`kubectl create configmap x --from-literal=K=v`	un test rapide
`kubectl create configmap x --from-file=cle=fichier`	un fichier entier sous une clé
`kubectl create secret generic x --from-literal=… --dry-run=client -o yaml`	**produire** le manifeste (avec le base64 calculé) sans le créer, pour le chiffrer ensuite

`--from-env-file=api.env` lit un fichier `CLE=valeur` du labo 08 : la transition de Compose vers Kubernetes se fait sans réécrire la configuration.

5. Ce qui change entre environnements

Le Deployment référence `api-config` et `db-secret` par leur **nom** ; chaque namespace (ou chaque cluster) possède ses propres objets de ce nom, avec ses valeurs. Promouvoir une image de recette vers production (labo 10) consiste à appliquer le **même** Deployment dans un namespace où les ConfigMaps et Secrets disent `prod`. Kustomize et Helm (labo 17) génèrent ces variantes ; le mécanisme sous-jacent reste celui-ci.

> K3S
> Rien de spécifique : ConfigMaps et Secrets sont dans le cœur de l'API, et k3s stocke les Secrets en clair dans SQLite comme un cluster nu les stocke dans etcd. L'option `--secrets-encryption` de k3s active le chiffrement au repos ; un cluster managé le propose en case à cocher. Sur votre poste, ce n'est pas le sujet : la base `k3s.db` est à vous.

6. En entreprise

- **Une ConfigMap par application et par environnement**, versionnée avec les manifestes ; les valeurs qui diffèrent (`SPRING_PROFILES_ACTIVE`, URL de base, niveaux de log) y vivent,

jamais dans le template.

- **Les Secrets sont générés, pas écrits** : External Secrets Operator ou un équivalent les synchronise depuis le gestionnaire de secrets de l'entreprise ; le dépôt Git ne contient que la *référence* (« le mot de passe de la base `app` en prod »).
- **Fichiers pour les secrets** (`volume: secret`, convention `_FILE` / `configtree`), variables pour la configuration : la même discipline qu'au labo 08, parce que `kubectl exec … env` montre tout l'environnement.
- **Rotation** : un Secret monté en volume se met à jour sans redémarrage, à condition que l'application relise le fichier ; un Secret en variable exige un `rollout restart`. C'est un critère de conception, pas un détail.
- **RBAC** : lire les ConfigMaps d'un namespace est courant ; lire ses Secrets est réservé.

À retenir

- ConfigMap = configuration non sensible, Secret = accès ; les deux sont des dictionnaires référencés par le template, par nom.
- Variables (`envFrom`, `valueFrom`) pour ce qui est lu au démarrage ; volumes pour les fichiers de configuration et les secrets.
- Modifier une ConfigMap ne change pas les variables des Pods en cours (`rollout restart`) ; un fichier monté se met à jour seul, sauf avec `subPath`.
- Une clé manquante bloque le Pod : `CreateContainerConfigError`, `couldn't find key`.
- Base64 n'est pas un chiffrement ; un Secret se protège par RBAC, chiffrement au repos, et en ne le commitant jamais en clair.
- `--dry-run=client -o yaml` produit un manifeste ; `--from-env-file` reprend un `api.env` du labo 08.
- Même Deployment partout, ConfigMaps et Secrets différents par namespace : c'est ainsi qu'une image est promue.

Vocabulaire

ConfigMap : dictionnaire de configuration non sensible. — **Secret** : dictionnaire de valeurs sensibles, encodées en base64, montées en tmpfs. — **`envFrom`** / **`valueFrom`** : injection de toutes les clés / d'une clé en variable. — **volume** **`configMap`** / **`secret`** : injection en fichiers, un par clé. — **`subPath`** : montage d'un seul fichier, sans mise à jour automatique. — **`CreateContainerConfigError`** : clé ou objet référencé introuvable. — **`stringData`** / **`data`** : valeurs en clair / en base64 dans un Secret. — **`immutable`** : objet non modifiable. — **chiffrement au repos** : protection des Secrets dans etcd. — **External Secrets / Sealed Secrets / SOPS** : outils pour ne jamais commiter un secret en clair.